

Website Flow & Build Guide

TigerWatch Platform Automated Camera Trap Triage & Tiger Movement Intelligence System

Pench Tiger Reserve

Document Version: 1.0
Date: August 15, 2026
Type: Website Flow, Navigation Architecture & Build Guide
Purpose: Step-by-step construction guide for AI agents & developers
Companion Docs: PRD v1.0, TRD v1.0, UI/UX Design System v1.0
App Name: TigerWatch
Tech Stack: Next.js 14, React 18, Leaflet.js, D3.js, FastAPI, PostgreSQL

Table of Contents

- 1. Application Overview & Sitemap
- 2. Route Structure & URL Schema
- 3. Global Layout & Navigation Flow
- 4. Authentication Flow
- 5. Page-by-Page Build Specification
 - 5.1 Login Page
 - 5.2 Dashboard (Home)
 - 5.3 Processing: Upload & Run
 - 5.4 Processing: Run Monitor
 - 5.5 Tiger Catalogue (Grid/List)
 - 5.6 Tiger Individual Profile
 - 5.7 Image Review Queue
 - 5.8 Occupancy Map
 - 5.9 Alert Feed
 - 5.10 Alert Detail
 - 5.11 Station Management
 - 5.12 Settings & Admin
- 6. User Journeys (End-to-End Flows)
 - 6.1 Journey A: Full Survey Processing
 - 6.2 Journey B: Respond to Critical Alert
 - 6.3 Journey C: Investigate Individual Tiger
 - 6.4 Journey D: Spatial Analysis & Export
- 7. State Management & Data Flow
- 8. Component Hierarchy & Build Order
- 9. API Integration Map
- 10. Error States & Empty States
- 11. File & Folder Structure
- 12. Build Sequence (Step-by-Step)

1. Application Overview & Sitemap

TigerWatch is a single-page application (SPA) built with Next.js 14 (App Router). It consists of 12 primary pages organized under an authenticated shell layout. The application name is 'TigerWatch' and the tagline is 'Pench Tiger Reserve Intelligence Platform'.

Visual sitemap (hierarchical):

TigerWatch (Root)	
-- /login	[Public - Auth Page]
-- / (Authenticated Shell)	[Layout: Sidebar + TopBar + Content]
-- /dashboard	[Home - KPI Overview]
-- /processing	
-- /upload	[Upload images & start run]
-- /runs	[Run history list]
-- /runs/[runId]	[Live run monitor / run detail]
-- /catalogue	
-- /	[Tiger grid/list view]
-- /[tigerId]	[Individual tiger profile]
-- /overview	[Tab: summary stats + mini map]
-- /captures	[Tab: capture history table]
-- /map	[Tab: individual range map]
-- /alerts	[Tab: alerts for this tiger]
-- /review	[Image review queue]
-- /map	[Full occupancy map view]
-- /alerts	
-- /	[Alert feed list]
-- /[alertId]	[Alert detail page]
-- /stations	
-- /	[Station list + map]
-- /[stationId]	[Station detail]
-- /settings	
-- /general	[System configuration]
-- /users	[User management]
-- /models	[AI model configuration]
-- /alerts	[Alert rule configuration]

[BUILD PRIORITY]

Build pages in this order: Login > Layout Shell > Dashboard > Processing Upload > Catalogue Grid > Tiger Profile > Map > Alerts > Review Queue > Stations > Settings. Each page should be functional before moving to the next.

2. Route Structure & URL Schema

All routes follow Next.js 14 App Router conventions. Dynamic segments use [param] syntax.

Route	Component	Access	Primary API	Purpose
/login	LoginPage	Public	POST /auth/login	User authentication
/dashboard	DashboardPage	All	GET /dashboard/stats	Home, KPI cards, charts
/processing/upload	UploadPage	Admin, Bio	POST /filtering/run	Upload images, configure, start
/processing/runs	RunListPage	All	GET /runs	Processing run history
/processing/runs/[id]	RunDetailPage	All	GET /runs/{id}	Live monitor or completed detail
/catalogue	CataloguePage	All	GET /catalogue	Tiger grid/list with filters
/catalogue/[id]	TigerProfilePage	All	GET /catalogue/{id}	Individual profile with tabs
/review	ReviewQueuePage	Admin, Bio	GET /review/queue	Ambiguous match review
/map	OccupancyMapPage	All	GET /occupancy/*	Full-screen reserve map
/alerts	AlertFeedPage	All	GET /alerts	Filterable alert feed
/alerts/[id]	AlertDetailPage	All	GET /alerts/{id}	Alert evidence & actions
/stations	StationListPage	All	GET /stations	Station table + map
/stations/[id]	StationDetailPage	All	GET /stations/{id}	Station captures & health
/settings/general	SettingsPage	Admin	GET/PUT /config	Thresholds, system config
/settings/users	UserMgmtPage	Admin	GET /users	RBAC user management
/settings/models	ModelConfigPage	Admin	GET /models	AI model versions
/settings/alerts	AlertRulesPage	Admin	PUT /alerts/config	Alert rule thresholds

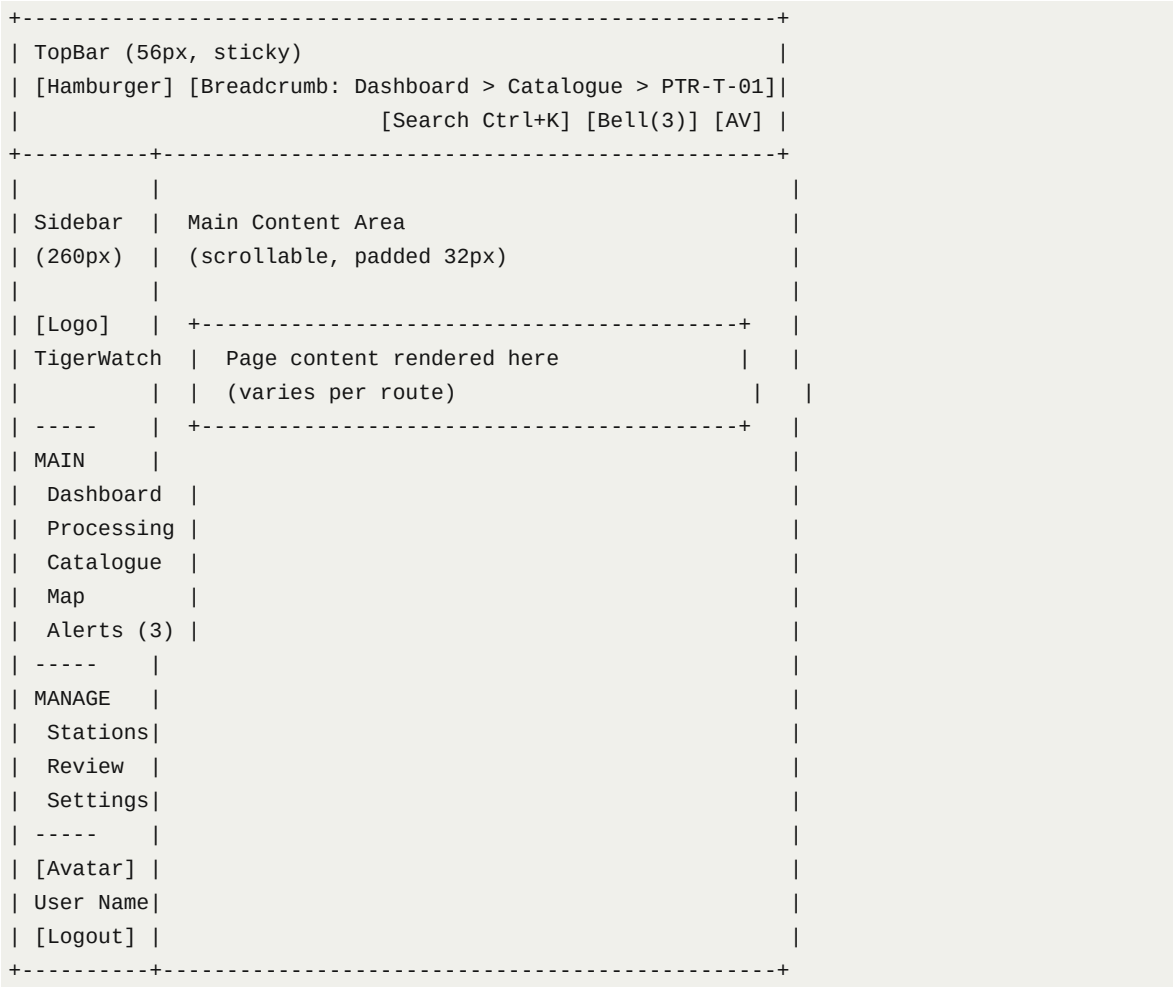
2.1 URL Parameter Conventions

- > Tiger IDs in URLs: use the tiger_id (e.g., PTR-T-001) not the UUID. URL: /catalogue/PTR-T-001
- > Run IDs: use short UUID prefix (8 chars). URL: /processing/runs/a1b2c3d4
- > Query parameters for filters: /alerts?type=BUFFER_APPROACH&confidence=HIGH&status=new
- > Pagination: /catalogue?page=2&per_page=20
- > Tab selection on profile: /catalogue/PTR-T-001?tab=captures

3. Global Layout & Navigation Flow

3.1 Authenticated Shell Layout

All authenticated pages share a common shell layout with three zones:



3.2 Navigation Items & Icons

- > Dashboard - icon: LayoutDashboard - route: /dashboard - always visible
- > Processing - icon: Cpu - route: /processing/upload - submenu: Upload, Run History
- > Catalogue - icon: PawPrint - route: /catalogue - badge: total tiger count
- > Map - icon: MapPinned - route: /map
- > Alerts - icon: Bell - route: /alerts - badge: unread alert count (red if BUFFER_APPROACH)
- > Stations - icon: Radio - route: /stations - section: MANAGE
- > Review Queue - icon: ScanEye - route: /review - badge: pending review count - section: MANAGE
- > Settings - icon: Settings - route: /settings/general - section: MANAGE - admin only

3.3 Navigation State Rules

- > Active nav item: highlighted with accent background + left border bar (3px amber).
- > Collapsed sidebar (tablet): icon-only mode (60px wide). Tooltip on hover showing label.
- > Mobile (<768px): sidebar hidden. Hamburger menu in TopBar. Full-screen overlay menu on toggle.
- > Processing sub-menu: expands inline showing 'Upload' and 'Run History' children.
- > Settings sub-menu: expands showing General, Users, Models, Alert Rules. Visible only to Admin role.

3.4 Breadcrumb Logic

- > /dashboard -> 'Dashboard'
- > /catalogue -> 'Dashboard > Catalogue'
- > /catalogue/PTR-T-001 -> 'Dashboard > Catalogue > PTR-T-001 (Rajaji)'
- > /alerts/abc123 -> 'Dashboard > Alerts > BUFFER_APPROACH #abc123'
- > /processing/runs/xyz -> 'Dashboard > Processing > Run #xyz'
- > Each segment is a clickable link back to that level.

3.5 Global Search (Cmd+K)

- > Trigger: click search icon in TopBar or press Ctrl+K / Cmd+K.
- > Opens: command palette modal (centered, 600px wide, max-height 500px).
- > Search across: tiger IDs/names, station codes/names, alert titles, run IDs.
- > Results grouped by type with icons. Arrow key navigation. Enter to navigate to result.
- > Recent searches shown when empty. Quick actions: 'Start new run', 'View map', 'Open settings'.

4. Authentication Flow

Complete authentication flow from first visit to authenticated session:

[**VISIT /**]

User navigates to TigerWatch. Next.js middleware checks for valid JWT in cookies.

|

v No valid JWT found

[**REDIRECT /login**]

User sees login page. Dark themed with TigerWatch logo, ambient background blobs, centered login card.

|

v User enters credentials

[**POST /api/v1/auth/login**]

Frontend sends { username, password } to API. Shows loading spinner on button.

|

v API returns { access_token, refresh_token, user }

[**STORE TOKENS**]

access_token stored in httpOnly cookie (1hr expiry). refresh_token in httpOnly cookie (7d). User object stored in React context.

|

v Redirect to /dashboard

[**DASHBOARD LOADS**]

Shell layout renders. Sidebar, TopBar, and Dashboard content load. Initial data fetched.

Token Refresh Flow

- > When API returns 401: frontend interceptor calls POST /api/v1/auth/refresh with refresh_token.
- > If refresh succeeds: new access_token set, original request retried automatically.
- > If refresh fails (expired/invalid): user redirected to /login with message 'Session expired. Please log in again.'
- > Auto-refresh: proactive refresh when access_token has < 5 minutes remaining (checked on each API call).

Logout Flow

- > User clicks Logout in sidebar bottom area.
- > POST /api/v1/auth/logout called. Tokens cleared from cookies. React context reset.
- > Redirect to /login with success toast 'Logged out successfully'.

Role-Based UI Gating

- > ADMIN: sees all nav items including Settings, User Management. Can start runs, purge data, manage models.
- > BIOLOGIST: sees all except User Management. Can approve reviews, manage catalogue, configure alert rules.
- > RANGE_OFFICER: sees Dashboard, Catalogue (read-only), Map, Alerts (can acknowledge/resolve), Stations (read-only).

- > FIELD_STAFF: sees Dashboard (limited), Upload page, Alerts (view only, assigned alerts). No GPS coordinates shown.
- > VIEWER: read-only access to Dashboard, Catalogue, Map, Alerts. No export, no GPS.
- > Unauthorized nav items: completely hidden (not grayed out). API returns 403 if role-gated endpoint called.

5. Page-by-Page Build Specification

Each page specification includes: route, layout, data sources, component breakdown, user interactions, state transitions, and edge cases. Build each page in the order listed.

5.1 Login Page (/login)

- >> **Layout:** Full-screen, no sidebar/topbar. Centered card (max-width 420px) on ambient background with floating gradient blobs.
- >> **Components:** Logo + tagline at top. Email input field. Password input field (with show/hide toggle). 'Sign In' primary button (full-width). Error message area below button.
- >> **Interactions:** Submit: validate fields (non-empty), call API, show loading state on button (spinner + 'Signing in...'). Success: redirect /dashboard. Failure: shake animation on card, red error message ('Invalid credentials' or 'Server error').
- >> **Edge Cases:** Rate limit (429): show 'Too many attempts. Try again in X seconds.' Network error: show 'Unable to connect. Check your connection.' Already authenticated: redirect to /dashboard immediately.

5.2 Dashboard (/dashboard)

- >> **Layout:** Sidebar + Content (Layout A). Content: stacked sections with bento grid.
- >> **Data Sources:** GET /api/v1/dashboard/stats (aggregated KPIs), GET /api/v1/alerts?status=new&limit=5 (recent alerts), GET /api/v1/runs?limit=5 (recent runs).
- >> **Section 1 - Welcome Bar:** Left: 'Welcome back, [Name]' (H2) + last run summary text. Right: 'Start New Run' primary button (links to /processing/upload). Full-width, no card container.
- >> **Section 2 - KPI Cards (Bento 4-col):** Card 1: 'Total Tigers' - large number (font-mono, gradient text), trend arrow + % vs last month. Card 2: 'Images This Month' - count with sparkline. Card 3: 'Active Alerts' - count, RED accent if > 0, links to /alerts. Card 4: 'Stations Online' - X/Y format, green progress ring.
- >> **Section 3 - Two-Column:** Left (col-span-2): Population trend line chart (D3.js, 12 months, accent color line). Right (col-span-1): Activity feed list - last 5 events (new tiger enrolled, alert raised, run completed) with icon + text + relative timestamp. Each clickable to source page.
- >> **Section 4 - Two-Column:** Left: Mini map (Leaflet, 350px height) showing all active tiger centroids as color-coded dots. Click dot navigates to /catalogue/[id]. Right: Alert breakdown donut chart (by type) + 'View All Alerts' link.
- >> **Section 5 - Recent Runs Table:** Table: 5 most recent processing runs. Columns: Run ID (mono), Date, Status (badge), Images, Tigers Found, Alerts. Click row navigates to /processing/runs/[id].
- >> **Refresh:** Auto-refresh every 60 seconds (configurable). Manual refresh button in top-right. Loading skeleton shown during refresh.

5.3 Processing: Upload & Run (/processing/upload)

- >> **Layout:** Sidebar + Content. Content: single-column, centered (max-width 700px).
- >> **Step 1 - Source Selection:** Card with two options: (A) 'Select Directory' - file picker for local directory path input. (B) 'Upload ZIP' - drag-and-drop zone (dashed border, icon, 'Drop files here or click to browse'). Max 10GB. Progress bar on upload.
- >> **Step 2 - Configuration:** After source selected, configuration panel slides in below: Blank threshold slider (0.05-0.50, default 0.2) with label showing current value. Auto-match threshold slider (0.50-0.95, default 0.85). Checkboxes: 'Run identification after filtering', 'Generate occupancy maps', 'Run alert detection'. Station mapping: dropdown to select method (auto-detect from EXIF / manual CSV upload / directory name pattern).
- >> **Step 3 - Confirm & Start:** Summary card: source path, estimated image count, selected options. 'Start Processing' primary button. 'Cancel' ghost button.
- >> **On Submit:** POST /api/v1/filtering/run with config. On 202 Accepted: redirect to /processing/runs/[newRunId] for live monitoring. On error: toast notification with error message.
- >> **Edge Cases:** No images found in directory: warning message. Upload interrupted: resume capability with progress retention. Active run already in progress: warning modal 'A run is already active. Queue this run?'

5.4 Processing: Run Monitor (/processing/runs/[runId])

- >> **Layout:** Sidebar + Content. Full-width content.
- >> **Active Run View:** Large pipeline visualization at top: horizontal stepper showing 6 stages: Ingesting > Filtering > Detecting > Matching > Analyzing > Alerting. Current stage highlighted with accent color + pulse animation. Completed stages: green checkmark. Pending: gray.
- >> **Live Stats Panel:** Below pipeline: 3-column grid. (1) Progress: circular gauge (% complete) + 'X / Y images processed' text + estimated time remaining. (2) Throughput: 'N images/sec' (font-mono, large) + GPU utilization bar. (3) Results so far: blanks removed, tigers detected, matches made, alerts found (counters animate up as processing progresses).
- >> **Live Updates:** WebSocket connection to /ws/runs/{runId}. Stats update in real-time (every 100 images). Progress bar animates smoothly. No page reload needed.
- >> **Completed Run View:** Pipeline shows all stages completed (green). Summary card replaces live stats: total images, blanks removed (% + storage saved), tigers detected, auto-matched, review-queued, new individuals, alerts generated. Action buttons: 'View Catalogue', 'View Alerts', 'View Map', 'Download Report (PDF)'.

5.5 Tiger Catalogue (/catalogue)

- >> **Layout:** Sidebar + Content. Content: filter bar + grid/list.
- >> **Filter Bar (top, sticky below TopBar):** Search input (placeholder: 'Search by ID or name...'). Status dropdown: All / Active / Absent / Provisional. Zone dropdown: All / Core / Buffer. Sort dropdown: Last Seen (default) / Name / Captures / Range Area. View toggle: Grid icon / List icon. Results count: 'Showing X of Y tigers'.
- >> **Grid View (default):** 4-column grid (desktop), 2 (tablet), 1 (mobile). Tiger Profile Cards (see UI/UX doc 5.2). Each card clickable - navigates to /catalogue/[tigerId].
- >> **List View:** Table with columns: [Thumbnail 40px] | Tiger ID (mono, accent) | Name | Status (badge) | Sex | Total Captures | Last Seen (relative) | Range (sq km). Sortable columns. Row click navigates to profile.
- >> **Pagination:** Bottom: 'Showing 1-20 of 62' + page numbers + per-page selector (20/50/100). Infinite scroll option on mobile.
- >> **Empty State:** If no tigers match filters: illustration + 'No tigers found' + 'Try adjusting your filters' + 'Clear filters' button.

5.6 Tiger Individual Profile (/catalogue/[tigerId])

- >> **Layout:** Sidebar + Content. Content: header + tabs.
- >> **Header Section:** Left: Tiger photo (200px square, rounded-xl, or placeholder paw icon if no photo). Right: Tiger ID (H3, font-mono, accent) + Assigned Name (H1, gradient text) + Status badge + Sex badge. Below: 'First seen: [date]' + 'Last seen: [date] ([X days ago])' + 'Total captures: [N]'. Edit button (pencil icon) for Admin/Bio to edit name/sex/status.
- >> **Tab Bar:** Horizontal tabs below header: Overview | Captures | Map | Alerts. Active tab: accent underline + accent text. Tab selection updates URL query param (?tab=captures) for shareable links.

Overview Tab (default)

- > Stats row: 4 small stat cards - Unique Stations, Range Area (sq km), Core Area (sq km), Days Since Last Capture.
- > Mini map: 300px height Leaflet map showing this tiger's KDE polygon + capture point markers + centroid star. Click 'Expand' to go to full Map tab.
- > Recent captures: 2x2 grid of most recent 4 capture thumbnails with date + station label. Click thumbnail for lightbox zoom.
- > Overlap section: list of overlapping individuals with overlap index value + mini profile chips (clickable to their profile).

Captures Tab

- > Paginated table: columns - [Thumbnail 60px] | Date | Station | Flank (L/R badge) | Confidence (color-coded mono) | Review Status (badge) | Actions (view full image).
- > Click thumbnail: lightbox modal with full-resolution image, EXIF data, zoom/pan controls, download button.
- > Filter: by station, by date range, by flank side, by review status.

Map Tab

- > Full-width Leaflet map. This tiger's KDE 95% polygon (filled, tiger's color) + KDE 50% polygon (hatched) + all capture locations as markers + centroid.
- > Other tigers' ranges shown as ghosted outlines (10% opacity) for spatial context.
- > Station markers visible. Click marker shows popup: station name, capture date at this station, thumbnail.

Alerts Tab

- > Chronological list of all alerts involving this individual. Same card format as alert feed. Click navigates to [/alerts/\[alertId\]](/alerts/[alertId]).

5.7 Image Review Queue (/review)

- >> **Layout:** Full-width focus layout (Layout C). No sidebar visible (or collapsed). Content centered, max-width 1100px.
- >> **Queue Header:** 'Review Queue' (H1) + 'X images pending review' + progress bar (reviewed / total this run). 'Skip to next run' if multiple runs have pending reviews.
- >> **Review Card (Main Content):** Left panel (55%): Query image - the new capture to be identified. Full flank crop displayed large (400px+). Below: capture metadata (station, date, flank side, quality score).
- >> **Right panel (45%):** Top-K Candidate Matches (default K=5), ranked by similarity. Each candidate: thumbnail (150px) + Tiger ID (mono, accent) + Name + Similarity score (font-mono, color-coded: green ≥ 0.85 , amber 0.60-0.85, red < 0.60) + 'Last seen [date]' + capture count.
- >> **Comparison Mode:** Click any candidate: side-by-side flank view opens. Query flank | Candidate flank, same size. Opacity slider to blend/overlay the two images. This is the key interaction for human verification.
- >> **Actions (bottom bar, sticky):** 'Confirm Match' (primary, requires selecting a candidate first) + 'New Individual' (outlined, creates new enrollment) + 'Skip' (ghost, moves to next without deciding). Keyboard shortcuts: 1-5 select candidate, Enter confirm, N new, S skip. Shortcuts shown as hints.
- >> **After Action:** Card transitions out (slide left). Next review card slides in (from right). Progress bar updates. Toast: 'Matched to PTR-T-005' or 'New individual PTR-T-063 created'.
- >> **Empty State:** When queue is empty: celebration illustration + 'All caught up!' + 'No images pending review' + 'Back to Dashboard' button.

5.8 Occupancy Map (/map)

- >> **Layout:** Sidebar + Map + Panel (Layout B). Map takes 100% of remaining space. Right panel (360px) docked, collapsible.
- >> **Map (main area):** Full-height Leaflet map with dark tiles. Layers rendered: reserve boundary (core + buffer as dashed polygons), all camera stations (color-coded circle markers), KDE 95% polygons for visible tigers (color-coded, semi-transparent), centroids (star markers with labels).
- >> **Tiger Selector (in sidebar):** Scrollable list of all active individuals. Each row: color dot + Tiger ID + Name + checkbox. 'Select All / None' toggle at top. Filter input to search by ID/name. Checking/unchecking a tiger shows/hides their KDE polygon on map instantly.
- >> **Layer Control (top-right of map):** Checkboxes: Camera Stations, KDE 95% Ranges, KDE 50% Core Areas, Centroids, Overlap Zones, Buffer Boundary, Village Points. Each toggles the corresponding map layer.
- >> **Right Panel (detail dock):** Click any tiger on map or in selector: panel shows individual summary (photo, ID, name, range area, core area, centroid coords, capture count, last seen). 'View Full Profile' link. Overlap list: other tigers overlapping with this one + overlap index.
- >> **Map Interactions:** Click station marker: popup with station name, zone, total captures, top tigers captured

here. Click KDE polygon: selects that tiger, opens right panel. Zoom/pan standard Leaflet controls. Time slider (bottom): scrub through historical occupancy snapshots by run date.

>> **Export (in right panel):** Buttons: 'Export GeoJSON', 'Export Shapefile', 'Export KML', 'Export PDF Map'. Exports include all currently visible layers.

5.9 Alert Feed (/alerts)

- >> **Layout:** Sidebar + Content. Content: filter bar + alert card list.
- >> **Filter Bar:** Type dropdown: All / RANGE_SHIFT / NOVEL_STATION / BUFFER_APPROACH / PROLONGED_ABSENCE. Confidence: All / HIGH / MEDIUM / LOW. Status: All / New / Acknowledged / Resolved / False Positive. Date range picker. 'Clear Filters' link.
- >> **Alert Cards (list):** Each alert is a card in a vertical list. Left: colored bar (type color). Content: [Type Icon] + Title (e.g., 'Range Shift Detected - PTR-T-012') + Description snippet (1 line) + Tiger ID chip (clickable) + Confidence badge + Timestamp (relative). BUFFER_APPROACH cards: visually larger, pulsing red border if status = 'new'. Cards sorted: new first (then by confidence HIGH>MED>LOW, then by date desc).
- >> **Card Click:** Navigates to /alerts/[alertId] for full detail.
- >> **Bulk Actions:** Checkbox on each card. Select multiple > action bar appears at bottom: 'Acknowledge Selected' + 'Assign to...' dropdown.
- >> **Real-time:** WebSocket: new alerts appear at top of list with slide-in animation. Bell icon in TopBar updates count.

5.10 Alert Detail (/alerts/[alertId])

- >> **Layout:** Sidebar + Content. Content: single-column, max-width 900px.
- >> **Header:** Alert type icon (large, 32px, type color) + Title (H1) + Type badge + Confidence badge + Status badge + Timestamp. Tiger ID chip (clickable to profile).
- >> **Evidence Section:** Map: 400px height showing relevant locations (old range outline, new range/station, movement arrow). Below map: supporting images in thumbnail grid (click for lightbox). Measurement data: distance shifted (km), area change (sq km), direction bearing - all in font-mono with labels.
- >> **Timeline Section:** Chronological evidence trail: list of relevant captures at relevant stations with dates, building the case for the alert. Each entry: date + station + thumbnail + distance from baseline.
- >> **Actions (sticky bottom bar):** 'Acknowledge' (secondary) + 'Resolve' (primary) + 'Mark False Positive' (ghost) + 'Assign to' (dropdown). Notes text area: officer can add comments. All actions logged in audit trail.
- >> **Related Alerts:** Card list of other active alerts for the same individual, if any.

5.11 Station Management (/stations)

- >> **Layout:** Sidebar + Content. Split view: map (top, 400px) + table (bottom, scrollable).
- >> **Map View:** All stations plotted. Color: green (active core), amber (active buffer), red (village-adjacent), gray (inactive). Click marker: popup with station name, code, zone, last image date.
- >> **Table View:** Columns: Station Code (mono) | Name | Zone (badge) | GPS (mono, truncated) | Last Image | Total Captures | Status (dot). Sortable. Search input above.
- >> **Station Detail (/stations/[id]):** Header: station name + code + zone badge + GPS coords + map pin. Recent captures: thumbnail grid. Species breakdown: mini bar chart. Camera health: last maintenance date, total images, operational days. Edit metadata: Admin/Bio can update zone, name, status.

>> **Bulk Import:** 'Import Stations' button opens modal. CSV upload with format: code, name, latitude, longitude, zone. Preview table shown before confirming import. Validation: duplicate codes, out-of-bounds GPS, missing fields.

5.12 Settings & Administration (/settings/*)

- >> **Layout:** Sidebar + Content. Content: left sub-nav (vertical tabs) + settings panel.
- >> **General (/settings/general):** System configuration: blank threshold slider + current value preview. Auto-match threshold slider. Review threshold slider. Quarantine retention (days input). Batch size (number input). Each change: 'Save' button appears, confirms with toast on success.
- >> **Users (/settings/users):** User table: Name | Email | Role (dropdown to change) | Last Login | Status | Actions (disable/delete). 'Add User' button opens modal with form: name, email, temporary password, role selection. Admin only.
- >> **Models (/settings/models):** Cards for each model slot: Blank Detector (MegaDetector) | Tiger Detector | Embedding Extractor. Each card: model name, version, upload date, accuracy metrics, 'Upload New Version' button, 'Rollback' button. Version history accordion.
- >> **Alert Rules (/settings/alerts):** Per alert type: enable/disable toggle + threshold inputs. RANGE_SHIFT: core shift threshold (sq km input), buffer displacement (km input). NOVEL_STATION: distance threshold (km). BUFFER_APPROACH: always enabled (no threshold, just zone classification). PROLONGED_ABSENCE: required regularity score, grace period (cycles). Email recipients list. SMS recipients list (for HIGH priority).

6. User Journeys (End-to-End Flows)

6.1 Journey A: Full Survey Processing

Actor: Wildlife Biologist. Goal: Process a new batch of camera trap images from field collection.

[**1. LOGIN**]

Biologist logs in at /login. Redirected to /dashboard.

|

v Clicks 'Start New Run'

[**2. UPLOAD**]

At /processing/upload. Selects directory path (SD card contents). Configures thresholds. Enables all pipeline stages.

|

v Clicks 'Start Processing'

[**3. MONITOR**]

Redirected to /processing/runs/[id]. Watches live pipeline progress. Sees blanks being removed (70% filtered). Tigers detected. Matches accumulating.

|

v Run completes (45 min for 50K images)

[**4. REVIEW RESULTS**]

Summary card shows: 35K blanks removed, 340 tiger detections, 280 auto-matched, 42 pending review, 18 new individuals, 5 alerts. Clicks 'View Alerts'.

|

v Checks critical alerts first

[**5. ALERTS**]

At /alerts?status=new. Sees 1 BUFFER_APPROACH (HIGH) alert - PTR-T-023 detected near Sillari village. Clicks to review evidence.

|

v Alert is genuine. Acknowledges and assigns to Range Officer.

[**6. REVIEW QUEUE**]

Navigates to /review. Works through 42 ambiguous matches. Confirms 35, creates 7 new individuals. Queue empty.

|

v All images processed

[**7. MAP CHECK**]

Navigates to /map. Reviews updated occupancy for new individuals. Exports GeoJSON for QGIS analysis.

|

v Survey cycle complete

[**8. DASHBOARD**]

Returns to /dashboard. KPIs updated. Population trend shows +4 new individuals this quarter.

6.2 Journey B: Respond to Critical Alert

Actor: Range Officer. Goal: Respond to a buffer-zone intrusion alert.

[1. NOTIFICATION]

Officer receives SMS: 'CRITICAL: PTR-T-023 detected near Sillari buffer zone. Open TigerWatch for details.'
Also receives email with summary.

|

v Opens TigerWatch on tablet

[2. ALERT FEED]

At /alerts. BUFFER_APPROACH alert is top of list with pulsing red border. Clicks to open detail.

|

v Reviews evidence

[3. ALERT DETAIL]

Sees map with tiger's movement toward buffer zone. Views supporting images (3 captures at buffer stations over last 2 weeks). Distance: 2.3 km from nearest village.

|

v Determines action needed

[4. ACKNOWLEDGE]

Clicks 'Acknowledge'. Adds note: 'Deploying patrol team to Sillari sector. Alerting village sarpanch.' Assigns to self.

|

v Takes field action

[5. RESOLVE]

After field assessment (2 days later): returns to alert. Clicks 'Resolve'. Adds note: 'Tiger moved back to core zone. Warning issued to village. No conflict incident.'

6.3 Journey C: Investigate Individual Tiger

Actor: Biologist. Goal: Understand PTR-T-007's recent behaviour after noticing unusual captures.

[1. SEARCH]

Presses Cmd+K. Types 'PTR-T-007'. Clicks result to navigate to /catalogue/PTR-T-007.

|

v Views profile

[2. OVERVIEW TAB]

Sees stats: 47 total captures, range = 45 sq km, last seen 3 days ago. Mini map shows KDE polygon. Notes range seems larger than expected.

|

v Switches to Map tab

[3. MAP TAB]

Full map view. KDE polygon visible. Notices captures at 2 new stations in the southeast. Clicks markers to see dates and images.

|

v Checks captures for detail

[4. CAPTURES TAB]

Filters by last 30 days. Sees 8 new captures, 3 at previously unvisited stations. Reviews images - confirms

good quality matches.

|

v Checks alerts

[5. ALERTS TAB]

Sees RANGE_SHIFT alert from last run (MEDIUM confidence). Reviews and confirms the shift is genuine. Makes note in research log.

6.4 Journey D: Spatial Analysis & Export

Actor: Biologist. Goal: Export occupancy data for quarterly NTCA report.

[1. MAP VIEW]

Navigates to /map. Selects all active tigers in sidebar. Enables all layers.

|

v Adjusts view

[2. CONFIGURE]

Enables KDE 95%, KDE 50%, centroids, overlap zones. Hides stations for clean view.

|

v Exports data

[3. EXPORT]

Clicks 'Export Shapefile' in right panel. Downloads ZIP with all polygons + metadata. Clicks 'Export PDF Map' for printable version with legend.

[4. DASHBOARD]

Returns to /dashboard. Downloads processing summary report for the quarter from recent runs table.

7. State Management & Data Flow

7.1 Client-Side State Architecture

State management uses React Context + useReducer for global state, with SWR (or TanStack Query) for server state caching:

- >> **AuthContext:** User object, JWT tokens, role, login/logout actions. Persisted in httpOnly cookies. Checked by middleware on every route change.
- >> **UIContext:** Sidebar collapsed state, active theme, notification preferences, mobile menu open state. Persisted in localStorage.
- >> **Server State (SWR):** All API data fetched and cached by SWR with stale-while-revalidate strategy. Cache keys match API URLs. Automatic revalidation on window focus. Manual revalidation triggered after mutations (e.g., after confirming a review match, revalidate catalogue and review queue).

7.2 Real-Time Data (WebSocket)

- > WebSocket connection established on authenticated shell mount. Single connection, multiplexed channels.
- > Channels: (1) processing-progress: live run stats updates. (2) new-alerts: instant alert notifications. (3) system-health: GPU/memory/disk status.
- > On new alert message: TopBar bell badge increments + toast notification + if user is on /alerts page, new card inserts at top with animation.
- > On processing progress: if user is on /processing/runs/[id], live counters and progress bar update. If on /dashboard, processing indicator in TopBar pulses.
- > Connection lost: subtle amber banner 'Reconnecting...' + automatic retry with exponential backoff (1s, 2s, 4s, 8s, max 30s).

7.3 Data Mutation Flow

Standard pattern for all write operations:

- > 1. User action triggers mutation (button click, form submit, drag-drop).
- > 2. Optimistic UI update (where safe): UI reflects expected state immediately.
- > 3. API call fires (POST/PUT/DELETE).
- > 4. On success: SWR cache invalidated for affected resources. Toast: 'Action completed'.
- > 5. On failure: optimistic update rolled back. Error toast with message. If 401, trigger token refresh flow.

8. Component Hierarchy & Build Order

8.1 Atomic Design Levels

Components organized in atomic design hierarchy. Build bottom-up:

Atoms (build first)

- > Button (primary/secondary/ghost/danger/icon variants)
- > Input (text/search/number/password)
- > Select (single-select dropdown)
- > Badge (status/confidence/count)
- > StatusDot (active/absent/offline + optional pulse)
- > Slider (threshold control)
- > Toggle (on/off switch)
- > Tooltip (hover info)
- > Spinner (loading indicator)
- > Avatar (user photo/initials)
- > Icon (Lucide icon wrapper with size/color props)

Molecules (build second)

- > StatCard (label + value + trend + optional sparkline)
- > TigerCard (thumbnail + ID + name + status + captures)
- > AlertCard (type bar + icon + title + tiger chip + confidence + timestamp)
- > CaptureRow (thumbnail + date + station + flank + confidence + review status)
- > NavItem (icon + label + badge + active state)
- > FilterBar (search + dropdowns + sort + view toggle)
- > BreadcrumbTrail (clickable path segments)
- > EmptyState (illustration + message + action button)
- > Toast (icon + title + description + dismiss)

Organisms (build third)

- > Sidebar (logo + nav items + user section)
- > TopBar (hamburger + breadcrumb + search + notifications + user menu)
- > CommandPalette (search modal with grouped results)
- > DataTable (header + sortable columns + rows + pagination)
- > TigerGrid (filter bar + card grid + pagination)
- > AlertFeed (filter bar + alert card list + bulk actions)
- > ReviewPanel (query image + candidate list + comparison + actions)
- > MapView (Leaflet map + layer control + tiger selector + detail panel)
- > KPIRow (row of StatCards in bento grid)
- > ProcessingPipeline (stage stepper + live stats)
- > Modal (backdrop + container + header + body + footer)
- > ImageLightbox (full-res image + EXIF data + zoom/pan + nav)

Templates (build fourth)

- > AuthenticatedShell (sidebar + topbar + content slot)
- > SidebarContentLayout (Layout A)
- > SidebarMapPanelLayout (Layout B)
- > FullWidthFocusLayout (Layout C)

Pages (build last)

- > Each page composes organisms and molecules within a template. Pages handle data fetching (SWR hooks), state management, and route-specific logic.

9. API Integration Map

Maps each page to its required API calls. Use this to plan SWR hooks and data fetching:

Dashboard:

- GET /dashboard/stats
- GET /alerts?status=new&limit=5
- GET /runs?limit=5
- GET /catalogue/summary

Upload:

- POST /filtering/run
- GET /filtering/config

Run Monitor:

- GET /runs/{id}
- WS /ws/runs/{id}

Catalogue:

- GET /catalogue?page&status&zone&sort
- GET /catalogue/summary

Tiger Profile:

- GET /catalogue/{id}
- GET /catalogue/{id}/images
- GET /occupancy/individuals/{id}
- GET /alerts?individual_id={id}

Review Queue:

- GET /review/queue
- PUT /review/{id}
- POST /catalogue/{id}/merge

Occupancy Map:

- GET /occupancy/map/layers
- GET /occupancy/individuals/{id}
- GET /occupancy/overlap
- GET /stations

Alert Feed:

- GET /alerts?type&confidence&status&page

Alert Detail:

- GET /alerts/{id}
- PUT /alerts/{id}
- GET /catalogue/{individualId}

Stations:

- GET /stations
- GET /stations/{id}
- POST /stations (bulk import)

Settings:

- GET /filtering/config
- PUT /filtering/config
- GET /users
- POST /users
- GET /models
- PUT /alerts/config

10. Error States & Empty States

10.1 Error Handling Patterns

- >> **API Error (400/422):** Toast notification: red, with error message from API response. Form fields: highlight errored fields with red border + inline error message below field.
- >> **Unauthorized (401):** Silent token refresh attempt. If refresh fails: redirect to /login with message 'Session expired'.
- >> **Forbidden (403):** Toast: 'You do not have permission for this action.' No redirect - user stays on current page.
- >> **Not Found (404):** Full-page 404 component: 'Page not found' illustration + 'The page you're looking for doesn't exist.' + 'Back to Dashboard' button.
- >> **Server Error (500):** Full-page error component: 'Something went wrong' illustration + 'Our system encountered an error. Please try again.' + 'Retry' button + 'Contact Support' link.
- >> **Network Error:** Banner at top: amber, 'Connection lost. Retrying...' with subtle pulse animation. Auto-retry with exponential backoff. Banner disappears when connection restored.
- >> **Timeout:** Toast: 'Request timed out. Please try again.' + 'Retry' button in toast.

10.2 Empty State Designs

Every list/grid/table page must have a designed empty state:

- >> **Dashboard (first use):** Welcome illustration + 'Welcome to TigerWatch' + 'Start by uploading your first batch of camera trap images.' + 'Upload Images' primary button.
- >> **Catalogue (no tigers):** Tiger silhouette illustration + 'No tigers identified yet' + 'Process camera trap images to start building your catalogue.' + 'Start Processing' button.
- >> **Catalogue (filtered, no results):** Filter illustration + 'No tigers match your filters' + 'Try adjusting your search criteria.' + 'Clear Filters' button.
- >> **Alert Feed (no alerts):** Shield-check illustration + 'All clear!' + 'No alerts at this time. System is monitoring for deviations.' + muted text: 'Alerts are generated after each processing run.'
- >> **Review Queue (empty):** Celebration illustration + 'All caught up!' + 'No images pending review. Great work!' + 'Back to Dashboard' button.
- >> **Run History (no runs):** Processor illustration + 'No processing runs yet' + 'Upload camera trap images to start your first run.' + 'Upload Images' button.
- >> **Stations (no stations):** Map pin illustration + 'No stations configured' + 'Import your camera trap station locations to get started.' + 'Import Stations' button.

11. File & Folder Structure

Next.js 14 App Router structure. Every component has co-located CSS module.

```
tigerwatch/
|-- app/                                # Next.js App Router
|   |-- layout.tsx                      # Root layout (html, body, providers)
|   |-- globals.css                    # Global styles, CSS custom properties
|   |-- login/
|   |   |-- page.tsx                   # Login page
|   |   |-- login.module.css
|   |-- (authenticated)/               # Route group with auth middleware
|   |   |-- layout.tsx                 # Shell layout (sidebar + topbar + content)
|   |   |-- dashboard/
|   |   |   |-- page.tsx               # Dashboard page
|   |   |   |-- dashboard.module.css
|   |   |-- processing/
|   |   |   |-- upload/page.tsx
|   |   |   |-- runs/page.tsx
|   |   |   |-- runs/[runId]/page.tsx
|   |   |-- catalogue/
|   |   |   |-- page.tsx               # Catalogue grid/list
|   |   |   |-- [tigerId]/
|   |   |   |   |-- page.tsx          # Tiger profile (tabbed)
|   |   |   |   |-- profile.module.css
|   |   |-- review/page.tsx
|   |   |-- map/page.tsx
|   |   |-- alerts/
|   |   |   |-- page.tsx
|   |   |   |-- [alertId]/page.tsx
|   |   |-- stations/
|   |   |   |-- page.tsx
|   |   |   |-- [stationId]/page.tsx
|   |   |-- settings/
|   |       |-- layout.tsx             # Settings sub-nav layout
|   |       |-- general/page.tsx
|   |       |-- users/page.tsx
|   |       |-- models/page.tsx
|   |       |-- alerts/page.tsx
|
|-- components/
|   |-- atoms/
|   |   |-- Button/ (Button.tsx, Button.module.css)
|   |   |-- Input/ (Input.tsx, Input.module.css)
|   |   |-- Badge/ Select/ Slider/ Toggle/
|   |   |-- StatusDot/ Tooltip/ Spinner/ Avatar/
|   |-- molecules/
|   |   |-- StatCard/ TigerCard/ AlertCard/
|   |   |-- CaptureRow/ NavItem/ FilterBar/
|   |   |-- BreadcrumbTrail/ EmptyState/ Toast/
|   |-- organisms/
|   |   |-- Sidebar/ TopBar/ CommandPalette/
|   |   |-- DataTable/ TigerGrid/ AlertFeed/
```

```
| | |-- ReviewPanel/ MapView/ KPIRow/
| | |-- ProcessingPipeline/ Modal/ ImageLightbox/
| |-- templates/
| | |-- AuthenticatedShell/
|
|-- hooks/                                # Custom React hooks
| |-- useAuth.ts
| |-- useWebSocket.ts
| |-- useTigers.ts (SWR hook)
| |-- useAlerts.ts useRuns.ts useStations.ts
| |-- useOccupancy.ts useReviewQueue.ts
|
|-- lib/                                  # Utilities
| |-- api.ts                             # API client (fetch wrapper)
| |-- auth.ts                            # Token management
| |-- constants.ts                       # App constants, color palette
| |-- formatters.ts                      # Date, number, GPS formatters
| |-- mapUtils.ts                        # Leaflet helpers
|
|-- styles/
| |-- tokens.css                         # All CSS custom properties
| |-- animations.css                     # Keyframe animations
| |-- typography.css                     # Font imports + type styles
|
|-- public/
| |-- fonts/                             # Inter, JetBrains Mono (self-hosted)
| |-- icons/                             # Favicon, PWA icons
| |-- illustrations/                     # Empty state SVGs
```

12. Build Sequence (Step-by-Step)

Follow this exact sequence to build TigerWatch from scratch. Each step should produce a working, testable increment.

Phase 1: Foundation (Steps 1-4)

- >> **Step 1: Project Scaffold:** `npx create-next-app@latest ./ --typescript --app --no-tailwind --no-eslint`. Install dependencies: lucide-react, swr, leaflet, react-leaflet, d3, recharts. Set up folder structure as defined in Section 11.
- >> **Step 2: Design Tokens & Global Styles:** Create `styles/tokens.css` with ALL CSS custom properties from UI/UX doc Section 13. Create `globals.css` importing tokens, typography, animations. Import Inter and JetBrains Mono fonts. Set up ambient background (radial gradient + noise texture).
- >> **Step 3: Atoms:** Build all atom components: Button (5 variants), Input, Select, Badge, StatusDot, Slider, Toggle, Tooltip, Spinner, Avatar. Each with its CSS module. Test in isolation.
- >> **Step 4: Auth Flow:** Build `/login` page with auth form. Set up `AuthContext` with JWT management. Build `useAuth` hook. Create Next.js middleware for route protection. Test: login, redirect, token refresh, logout.

Phase 2: Shell & Navigation (Steps 5-7)

- >> **Step 5: Sidebar & TopBar:** Build Sidebar organism (logo, nav items, user section, collapsed mode). Build TopBar organism (hamburger, breadcrumb, search icon, notification bell, user menu). Wire navigation with Next.js router.
- >> **Step 6: Authenticated Shell:** Build `AuthenticatedShell` template combining Sidebar + TopBar + content area. Apply to `(authenticated) layout.tsx`. Test: navigation between placeholder pages, active states, sidebar collapse, mobile menu.
- >> **Step 7: Molecules:** Build remaining molecules: StatCard, TigerCard, AlertCard, CaptureRow, NavItem, FilterBar, BreadcrumbTrail, EmptyState, Toast. Toast notification system (context + stack).

Phase 3: Core Pages (Steps 8-12)

- >> **Step 8: Dashboard:** Build `DashboardPage`. Wire SWR hooks to API. KPI stat cards row. Population trend chart (D3.js or Recharts line chart). Activity feed list. Mini Leaflet map. Recent runs table. Auto-refresh. Empty state for first-time user.
- >> **Step 9: Processing Upload + Monitor:** Upload page: file picker, config sliders, start button. Run monitor page: pipeline stepper, live stats (wire WebSocket), progress gauge. Completed run summary. Run history list page.
- >> **Step 10: Tiger Catalogue + Profile:** Catalogue: filter bar + grid view + list view + pagination. Tiger profile: header, tab navigation (Overview, Captures, Map, Alerts), all tab content. Image lightbox for capture viewing.
- >> **Step 11: Occupancy Map:** Full MapView organism with Leaflet. Dark tiles. Reserve boundary GeoJSON overlay. Station markers. KDE polygon rendering (from GeoJSON API). Tiger selector in sidebar. Layer controls. Detail dock panel. Time slider. Export buttons.

- >> **Step 12: Alerts:** Alert feed page with filters, alert cards, real-time updates. Alert detail page with evidence map, timeline, action buttons. WebSocket integration for live alert push.

Phase 4: Supporting Pages (Steps 13-15)

- >> **Step 13: Review Queue:** ReviewPanel organism. Query image display. Candidate match list. Side-by-side flank comparison with opacity slider. Action buttons with keyboard shortcuts. Queue progress. Transition animations between reviews.
- >> **Step 14: Stations:** Station list page: map view (top) + table (bottom). Station detail page. Bulk CSV import modal with preview/validation.
- >> **Step 15: Settings:** Settings layout with sub-nav tabs. General config page (threshold sliders). User management table. Model config cards. Alert rule configuration forms.

Phase 5: Polish (Steps 16-18)

- >> **Step 16: Command Palette:** Build CommandPalette organism. Search across tigers, stations, alerts, runs. Grouped results. Keyboard navigation. Cmd+K trigger.
- >> **Step 17: Ambient Animations:** Add floating gradient blobs to background. Card hover effects (translateY, glow). Entrance animations (staggered fade-up). Map polygon draw animations. Reduced motion support.
- >> **Step 18: Responsive & Mobile:** Test all pages at 768px and 375px. Mobile hamburger menu. Bottom tab bar (mobile). Touch-optimized map controls. Swipe gestures on review/alerts. Offline indicator banner.

[TESTING AT EACH STEP]

After each step, verify: (1) page renders without errors, (2) navigation works, (3) responsive at all breakpoints, (4) API integration returns data (or mock data if backend not ready), (5) loading/empty/error states handled.

--- End of Document ---

Website Flow & Build Guide v1.0
TigerWatch - Pench Tiger Reserve Intelligence Platform

Follow the build sequence precisely. Each step produces a testable increment.